

SKILL WEEK – 6

Session – 11

TASK 1: To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
keerthi_priya@LAPTOP-OSGV × + ▾
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_COMMAND 200

int main()
{
    char history[MAX_HISTORY][MAX_COMMAND];
    char input[MAX_COMMAND];

    int count = 0;
    int next = 0;

    printf("Session 10 - Command History\n");
    printf("Enter commands (type 'history' to display, 'exit' to quit).\n\n");

    while (1)
    {
        printf("myshell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        /* Ignore empty commands */
        if (strlen(input) == 0)
            continue;

        /* Display history */
        if (strcmp(input, "history") == 0)
        {
            printf("\nCommand History:\n");

            if (count == 0)
            {
                printf("History is empty.\n");
            }
            else
            {
                int start = (count < MAX_HISTORY) ? 0 : next;

                for (int i = 0; i < count; i++)
                {
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
printf("\nCommand History:\n");

if (count == 0)
{
    printf("History is empty.\n");
}
else
{
    int start = (count < MAX_HISTORY) ? 0 : next;

    for (int i = 0; i < count; i++)
    {
        int index = (start + i) % MAX_HISTORY;

        printf("%d %s\n", i + 1, history[index]);
    }

    printf("\nHistory validation: SUCCESS\n");
    continue;
}

/* Exit */
if (strcmp(input, "exit") == 0)
{
    printf("\nExiting shell...\n");
    break;
}

/* Store command */
strcpy(history[next], input);

next = (next + 1) % MAX_HISTORY;

if (count < MAX_HISTORY)
    count++;

printf("Command stored in history.\n");
}

printf("\nHistory buffer contains %d command(s).\n", count);
printf("History consistency verified.\n");

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session11_task1.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session11_task1.c -o session11_task1
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session11_task1
Session 10 - Command History
Enter commands (type 'history' to display, 'exit' to quit).

myshell> ls
Command stored in history.
myshell> pwd
Command stored in history.
myshell> echo hello
Command stored in history.
myshell> history

Command History:
1  ls
2  pwd
3  echo hello

History validation: SUCCESS
myshell> exit

Exiting shell...

History buffer contains 3 command(s).
History consistency verified.
keerthi_priya@LAPTOP-OSGVCE7A:~$ |
```

TASK 2: To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_INPUT 500

typedef struct {
    char *args[MAX_ARGS];
    int argc;
} Command;

int parse_pipeline(char *input, Command commands[])
{
    int command_count = 0;

    char *command_part = strtok(input, "|");

    while (command_part != NULL && command_count < MAX_COMMANDS)
    {
        commands[command_count].argc = 0;

        char *arg = strtok(command_part, " \\t\\n");

        while (arg != NULL && commands[command_count].argc < MAX_ARGS - 1)
        {
            commands[command_count].args[
                commands[command_count].argc++
            ] = arg;

            arg = strtok(NULL, " \\t\\n");
        }

        commands[command_count].args[
            commands[command_count].argc
        ] = NULL;

        if (commands[command_count].argc == 0)
        {
            printf("Invalid empty command in pipeline.\\n");
            return -1;
        }
    }
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
{
    printf("Invalid empty command in pipeline.\n");
    return -1;
}

command_count++;

command_part = strtok(NULL, "|");
}

return command_count;
}

int main()
{
    char input[MAX_INPUT];
    Command commands[MAX_COMMANDS];

    printf("Session 10 - Pipeline Execution\n");
    printf("Enter a pipeline command:\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty pipeline.\n");
        return 0;
    }

    int command_count = parse_pipeline(input, commands);

    if (command_count <= 0)
    {
        printf("Pipeline validation failed.\n");
        return 1;
    }

    printf("\nPipeline Structure:\n");

    for (int i = 0; i < command_count; i++)
    {
        printf("Command %d: ", i + 1);
    }
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1

for (int i = 0; i < command_count; i++)
{
    printf("Command %d: ", i + 1);

    for (int j = 0; j < commands[i].argc; j++)
    {
        printf("%s ", commands[i].args[j]);
    }

    printf("\n");
}

printf("\nPipeline validation: SUCCESS\n");
printf("Execution order maintained.\n");

int pipes[MAX_COMMANDS - 1][2];
pid_t pids[MAX_COMMANDS];

for (int i = 0; i < command_count - 1; i++)
{
    if (pipe(pipes[i]) == -1)
    {
        perror("pipe failed");
        return 1;
    }
}

for (int i = 0; i < command_count; i++)
{
    pids[i] = fork();

    if (pids[i] < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pids[i] == 0)
    {
        /* Connect input from previous command */
        if (i > 0)
        {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }
    }
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
if (pids[i] == 0)
{
    /* Connect input from previous command */
    if (i > 0)
    {
        dup2(pipes[i - 1][0], STDIN_FILENO);
    }

    /* Connect output to next command */
    if (i < command_count - 1)
    {
        dup2(pipes[i][1], STDOUT_FILENO);
    }

    /* Close all pipe descriptors */
    for (int j = 0; j < command_count - 1; j++)
    {
        close(pipes[j][0]);
        close(pipes[j][1]);
    }

    execvp(commands[i].args[0], commands[i].args);

    perror("execvp failed");
    exit(EXIT_FAILURE);
}

/* Parent closes all pipes */
for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/* Wait for all processes */
for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("\nPipeline execution completed successfully.\n");

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session11_task2.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session11_task2.c -o session11_task2
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session11_task2
Session 10 - Pipeline Execution
Enter a pipeline command:
myshell> ls | wc -l

Pipeline Structure:
Command 1: ls

Pipeline validation: SUCCESS
Execution order maintained.
' info'          practical5          session8_task1
OSSP             practical_memory   session8_task1.c
OS_LAB           server_to_client   session8_task2
ProcessTask      session1-_task1     session8_task2.c
a.out            session10_task1     session9_task1
client_to_server session10_task1.c    session9_task1.c
file.txt         session10_task2     session9_task2
info.txt         session10_task2.c   session9_task2.c
memory_analysis  session11_task1     skill1
memory_analysis..save session11_task1.c   skill4
memory_analysis.c session11_task2     task2
numbers.txt     session11_task2.c   task2.c
p-6.c           session3_task1      tmp
p62             session3_task1.c    week3
practical3      session3_task1.c.save
practical4      session3_task2.c

Pipeline execution completed successfully.
```

```

keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session11_task2.c -o session11_task2
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session11_task2
Session 10 - Pipeline Execution
Enter a pipeline command:
myshell> echo hello world | wx -w

Pipeline Structure:
Command 1: echo hello world

Pipeline validation: SUCCESS
Execution order maintained.
hello world

Pipeline execution completed successfully.
keerthi_priya@LAPTOP-OSGVCE7A:~$ |

```

Session – 12

TASK – 1: To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t child1, child2;

    printf("Session 10 - Pipe and Stream Redirection\n");

    /* Create pipe */
    if (pipe(pipefd) == -1)
    {
        perror("pipe failed");
        return 1;
    }

    printf("Pipe created successfully.\n");

    /* Create first child */
    child1 = fork();

    if (child1 < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (child1 == 0)
    {
        /* First child writes to pipe */
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        execlp("echo", "echo", "Hello from child process", NULL);
    }
}

```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1

    execlp("echo", "echo", "Hello from child process", NULL);

    perror("exec failed");
    exit(EXIT_FAILURE);
}

/* Create second child */
child2 = fork();

if (child2 < 0)
{
    perror("fork failed");
    return 1;
}

if (child2 == 0)
{
    /* Second child reads from pipe */
    close(pipefd[1]);

    dup2(pipefd[0], STDIN_FILENO);

    close(pipefd[0]);

    execlp("wc", "wc", "-w", NULL);

    perror("exec failed");
    exit(EXIT_FAILURE);
}

/* Parent closes both pipe descriptors */
close(pipefd[0]);
close(pipefd[1]);

printf("Connected commands executing...\n");

/* Synchronize child processes */
waitpid(child1, NULL, 0);
waitpid(child2, NULL, 0);
```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1

/* Create second child */
child2 = fork();

if (child2 < 0)
{
    perror("fork failed");
    return 1;
}

if (child2 == 0)
{
    /* Second child reads from pipe */
    close(pipefd[1]);

    dup2(pipefd[0], STDIN_FILENO);

    close(pipefd[0]);

    execlp("wc", "wc", "-w", NULL);

    perror("exec failed");
    exit(EXIT_FAILURE);
}

/* Parent closes both pipe descriptors */
close(pipefd[0]);
close(pipefd[1]);

printf("Connected commands executing...\n");

/* Synchronize child processes */
waitpid(child1, NULL, 0);
waitpid(child2, NULL, 0);

printf("Data flow completed successfully.\n");
printf("Parent process synchronized with both children.\n");

return 0;
}
```

```

keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session12_task1.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session12_task1.c -o session12_task1
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session12_task1
Session 10 - Pipe and Stream Redirection
Pipe created successfully.
Connected commands executing...
4
Data flow completed successfully.
Parent process synchronized with both children.
keerthi_priya@LAPTOP-OSGVCE7A:~$ |

```

TASK – 2: To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_INPUT 500

typedef struct {
    char *args[MAX_ARGS];
    int argc;
} Command;

int parse_pipeline(char *input, Command commands[])
{
    int count = 0;

    char *part = strtok(input, "|");

    while (part != NULL && count < MAX_COMMANDS)
    {
        commands[count].argc = 0;

        char *arg = strtok(part, "\t\n");

        while (arg != NULL && commands[count].argc < MAX_ARGS - 1)
        {
            commands[count].args[commands[count].argc++] = arg;
            arg = strtok(NULL, "\t\n");
        }

        commands[count].args[commands[count].argc] = NULL;

        if (commands[count].argc == 0)
        {
            printf("Error: Empty command in pipeline.\n");
            return -1;
        }

        count++;
        part = strtok(NULL, "|");
    }

    return count;
}

int main()
{
    char input[MAX_INPUT];

```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
{
    char input[MAX_INPUT];
    Command commands[MAX_COMMANDS];

    printf("Session 10 - Multiple Pipe Execution\n");
    printf("Enter a long pipeline:\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty pipeline.\n");
        return 0;
    }

    int command_count = parse_pipeline(input, commands);

    if (command_count <= 0)
    {
        printf("Pipeline parsing failed.\n");
        return 1;
    }

    printf("\nPipeline Structure:\n");

    for (int i = 0; i < command_count; i++)
    {
        printf("Command %d: ", i + 1);

        for (int j = 0; j < commands[i].argc; j++)
        {
            printf("%s ", commands[i].args[j]);
        }

        printf("\n");
    }

    printf("\nPipeline validation: SUCCESS\n");
    printf("Number of commands: %d\n", command_count);

    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    /* Create multiple pipes */
    for (int i = 0; i < command_count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            return 1;
        }
    }
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
    for (int i = 0; i < command_count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe failed");
            return 1;
        }
    }

    /* Launch multiple processes */
    for (int i = 0; i < command_count; i++)
    {
        pids[i] = fork();

        if (pids[i] < 0)
        {
            perror("fork failed");
            return 1;
        }

        if (pids[i] == 0)
        {
            /* Connect input from previous pipe */
            if (i > 0)
            {
                dup2(pipes[i - 1][0], STDIN_FILENO);
            }

            /* Connect output to next pipe */
            if (i < command_count - 1)
            {
                dup2(pipes[i][1], STDOUT_FILENO);
            }

            /* Close all pipe descriptors in child */
            for (int j = 0; j < command_count - 1; j++)
            {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            execvp(commands[i].args[0], commands[i].args);

            perror("execvp failed");
            exit(EXIT_FAILURE);
        }
    }

    /* Parent closes all pipe descriptors */
    for (int i = 0; i < command_count - 1; i++)
    {
        close(pipes[i][0]);
    }
}
```

```
keerthi_priya@LAPTOP-OSGV X + v
GNU nano 8.7.1
    perror("fork failed");
    return 1;
}

if (pids[i] == 0)
{
    /* Connect input from previous pipe */
    if (i > 0)
    {
        dup2(pipes[i - 1][0], STDIN_FILENO);
    }

    /* Connect output to next pipe */
    if (i < command_count - 1)
    {
        dup2(pipes[i][1], STDOUT_FILENO);
    }

    /* Close all pipe descriptors in child */
    for (int j = 0; j < command_count - 1; j++)
    {
        close(pipes[j][0]);
        close(pipes[j][1]);
    }

    execvp(commands[i].args[0], commands[i].args);

    perror("execvp failed");
    exit(EXIT_FAILURE);
}

/* Parent closes all pipe descriptors */
for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/* Coordinate execution */
for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("\nAll processes completed.\n");
printf("All pipe resources cleaned up.\n");
printf("Long pipeline executed successfully.\n");

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session12_task2.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session12_task2.c -o session12_task2
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session12_task2
Session 10 - Multiple Pipe Execution
Enter a long pipeline:
myshell> ls | grep .c | wc -l

Pipeline Structure:
Command 1: ls

Pipeline validation: SUCCESS
Number of commands: 1
' info'
OSSP          practical_memory  session3_task1.c.save
OS_LAB        server_to_client  session3_task2.c
session1-_task1 session8_task1
ProcessTask   session10_task1  session8_task1.c
a.out         session10_task1.c session8_task2
client_to_server session10_task2  session8_task2.c
file.txt      session10_task2.c session9_task1
info.txt      session11_task1  session9_task1.c
memory_analysis session11_task1.c session9_task2
memory_analysis..save session11_task2  session9_task2.c
memory_analysis.c session11_task2.c skill1
numbers.txt   session12_task1 skill4
p-6.c         session12_task1.c task2
p62           session12_task2  task2.c
practical3    session12_task2.c tmp
practical4    session3_task1   week3
practical5    session3_task1.c

All processes completed.
All pipe resources cleaned up.
Long pipeline executed successfully.
keerthi_priya@LAPTOP-OSGVCE7A:~$ |
```